



UNIVERSITÀ DI PARMA

Edges Detection Lines Detection

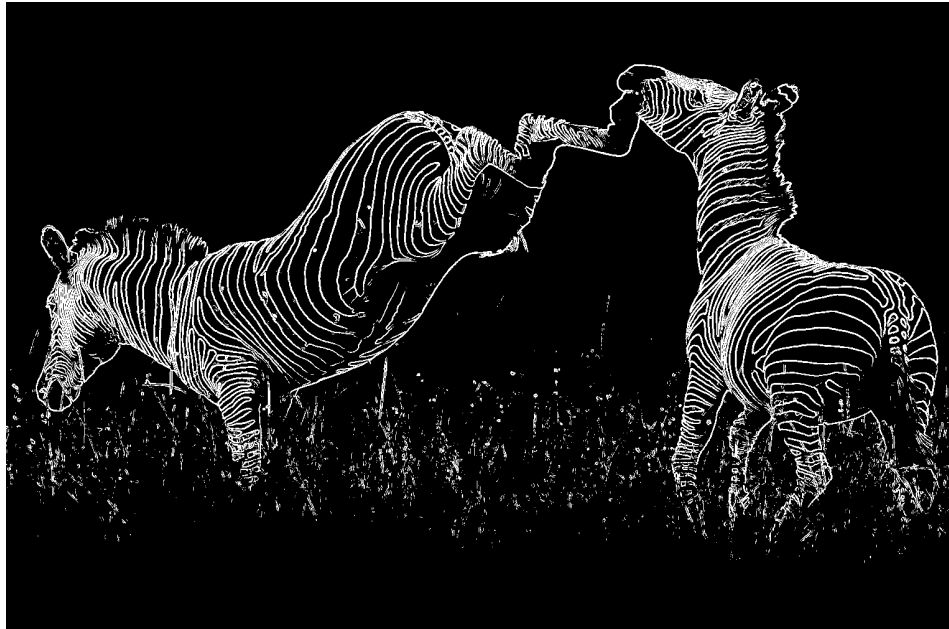
- Edge detection
 - What is an edge?
 - Are they important?
 - Canny Edge Detector
- Lines
 - Hough Transform
 - Generalized Hough Transform



UNIVERSITÀ DI PARMA

Edges Detection Lines Detection

- Q: What is an edge? A: A discontinuity in intensity!



Edge detection

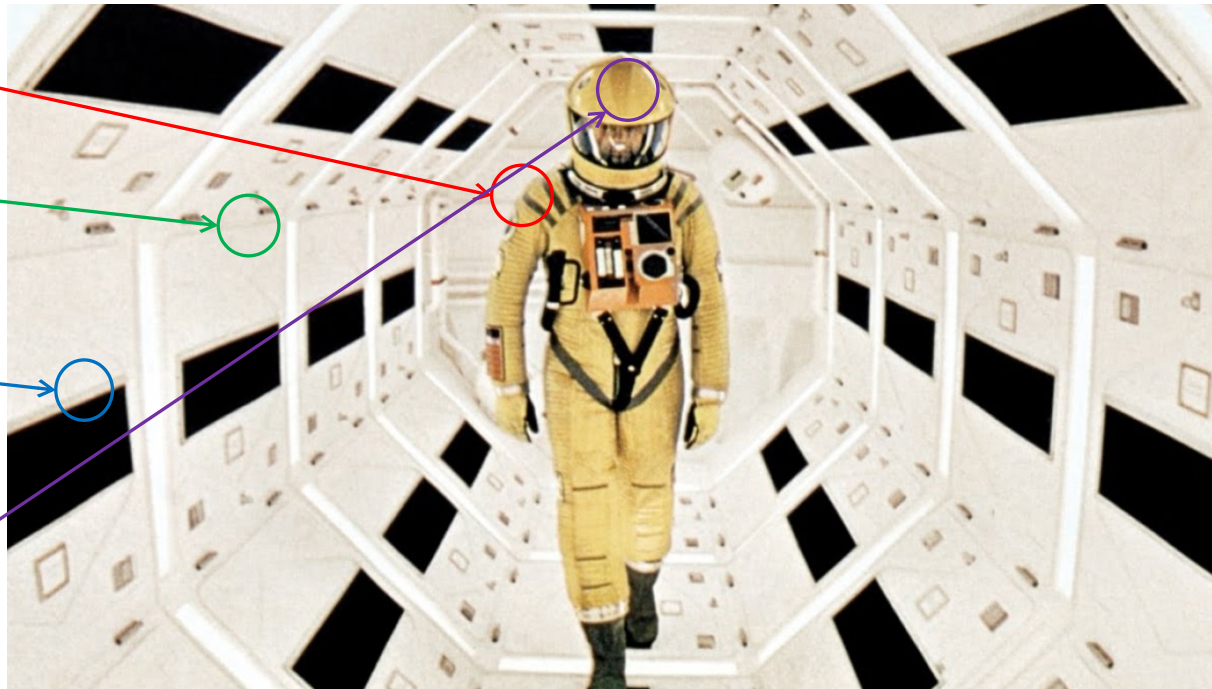
- Q: From where intensity discontinuities (aka edges) come from?

Depth differences

Orientations of surfaces

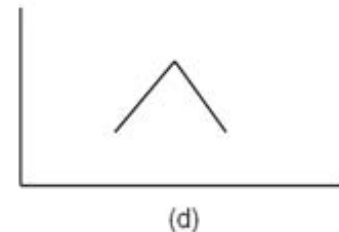
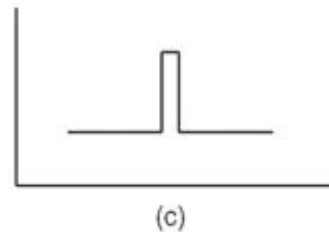
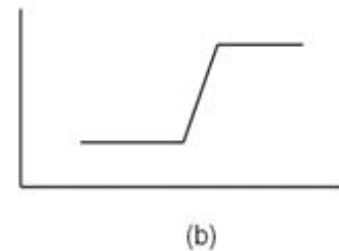
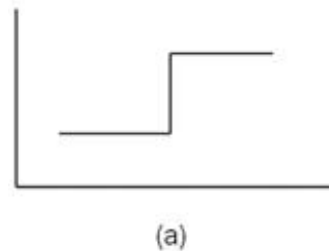
Properties of surfaces

Different illuminations (i.e. shadows or lights)



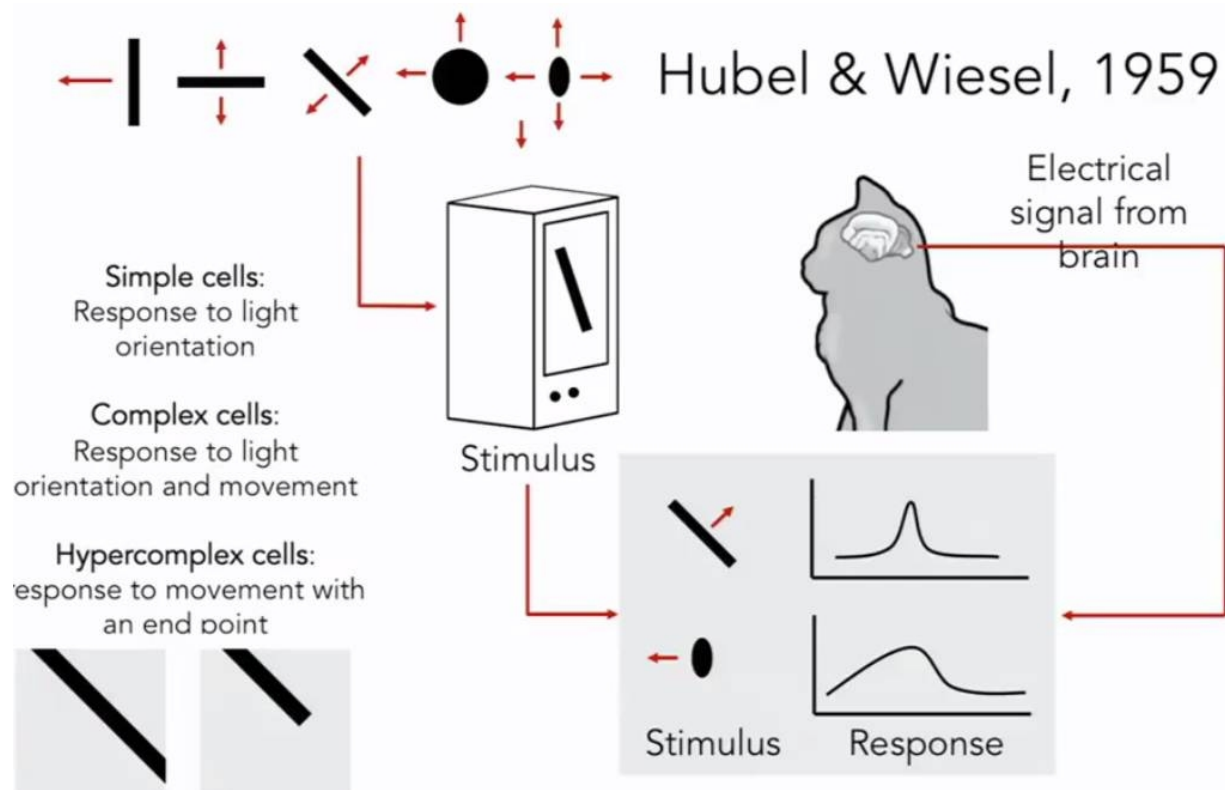
Types of edges

- (a) Step edge: abrupt change of intensity
- (b) Ramp edge: smooth change
- (c) Ridge/line edge: abrupt change followed by a return to starting value
- (d) Roof edge: a ridge edge with a smoother change

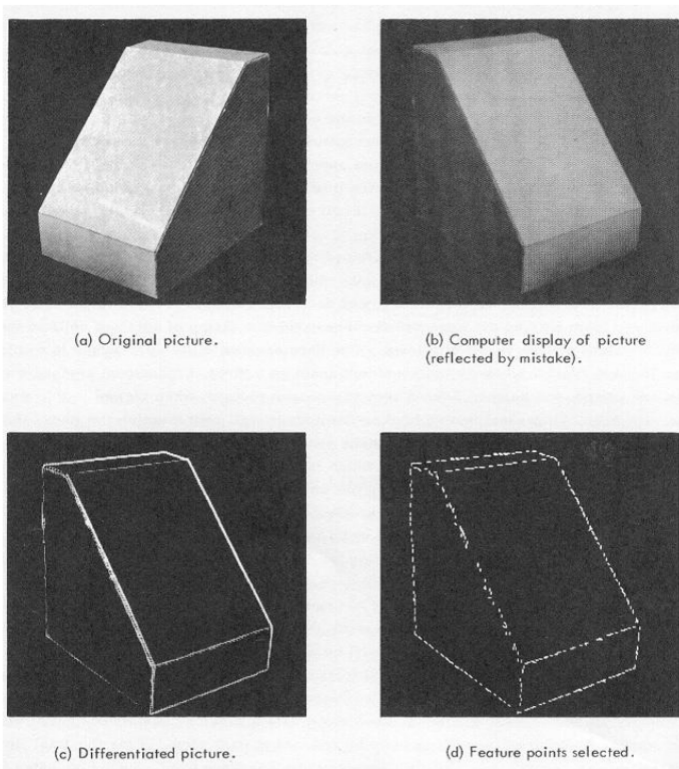


Edge Importance

- Why edges are important?



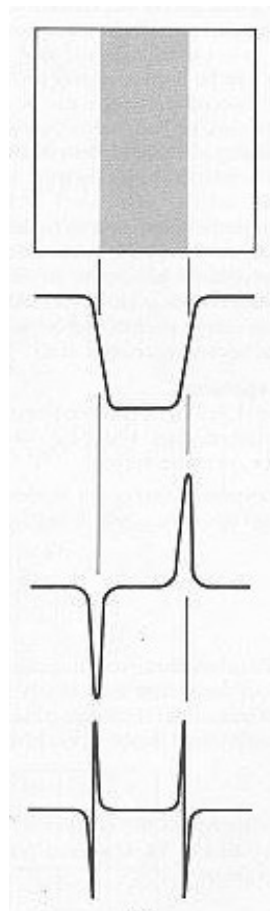
- Why edges are important?



Larry Roberts, 1963

- Edges can help:
 - Extract information
 - Detect objects
 - Detect object shapes

- Already (partially) discussed
 - Actually we discussed about gradient extraction
 - And gradient $\neq$ edges
- Small recap:
 - Smoothing $\rightarrow$ to reduce noise
 - Derivative Horizontal & Vertical filtering $\rightarrow$ extract gradient



Image

Horizontal Intensity

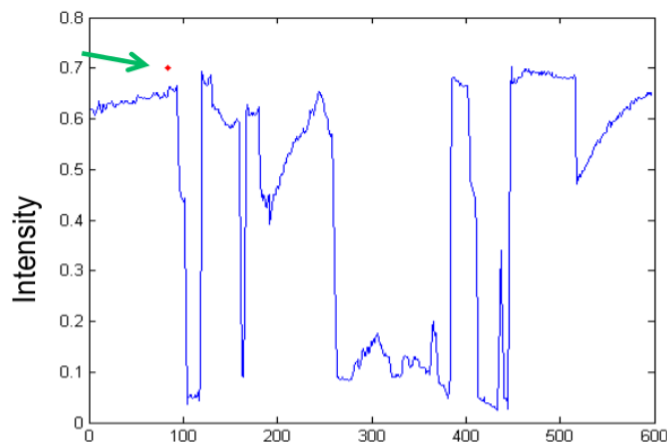
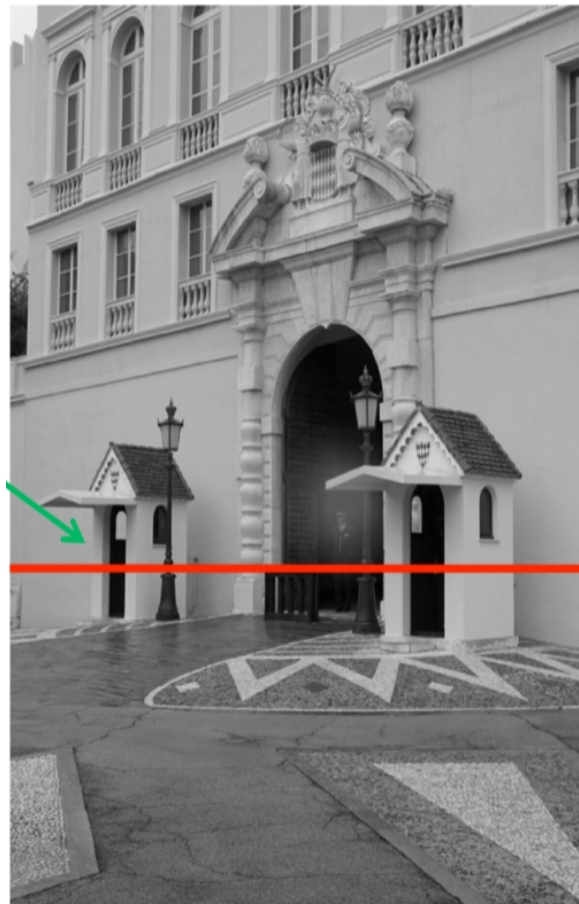
First derivative

Second derivative

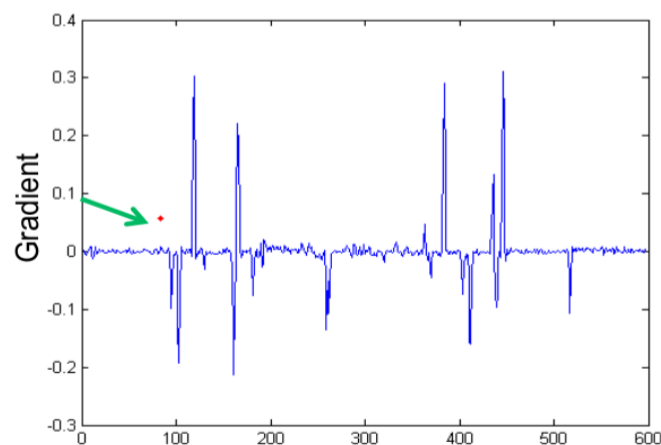
Edges are the maximum/minimum
in first derivative

Edges corresponds to 0 crossings in
second derivative

Gradient in real images



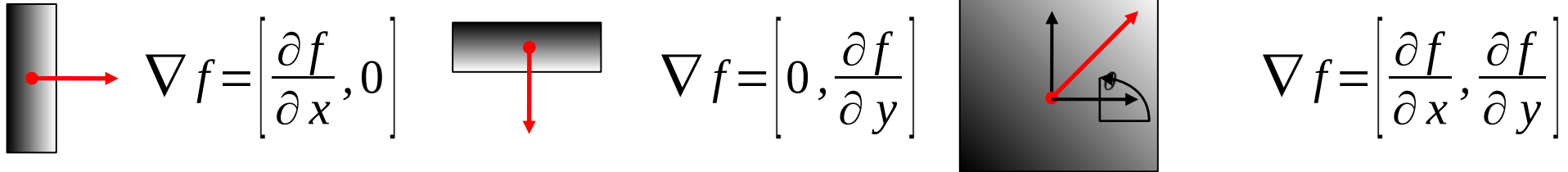
We have large peaks but also some noise



From: Fei Fei Li

How we can compute gradient?

- Gradient of image:



- Gradient is a vector:
 - Magnitude/Intensity
 - Direction

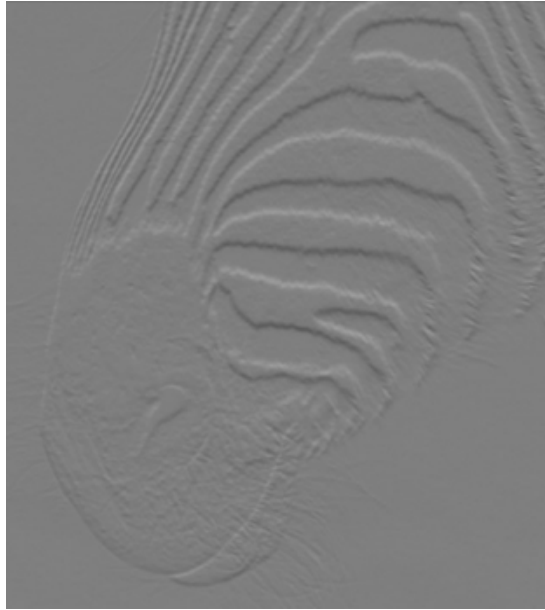
$$\|\nabla f\| = \sqrt{\left(\frac{\partial f}{\partial x}\right)^2 + \left(\frac{\partial f}{\partial y}\right)^2}$$

$$\theta = \tan^{-1}\left(\frac{\partial f}{\partial y} / \frac{\partial f}{\partial x}\right)$$

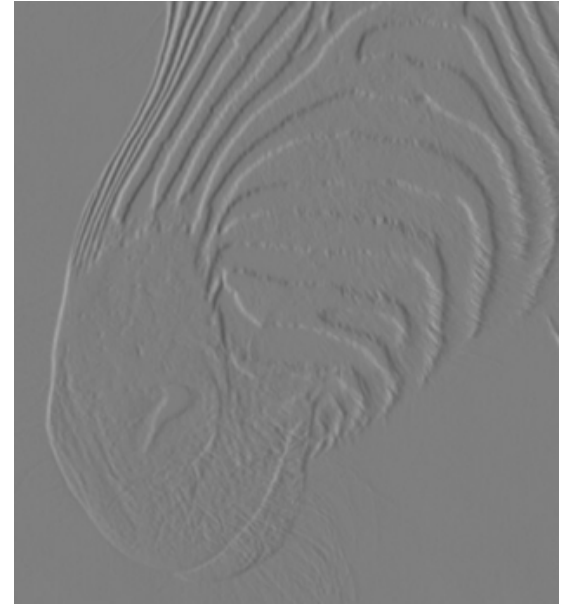
Example



f



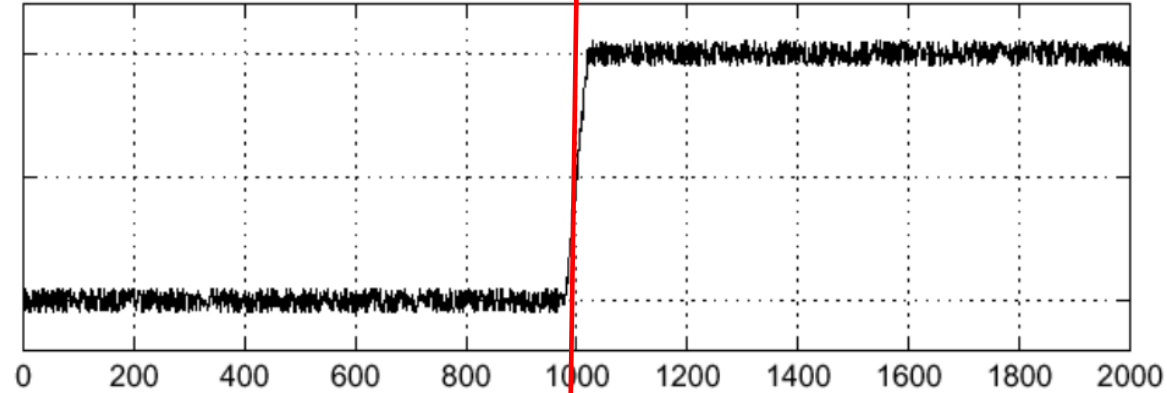
$\frac{\partial f}{\partial y}$



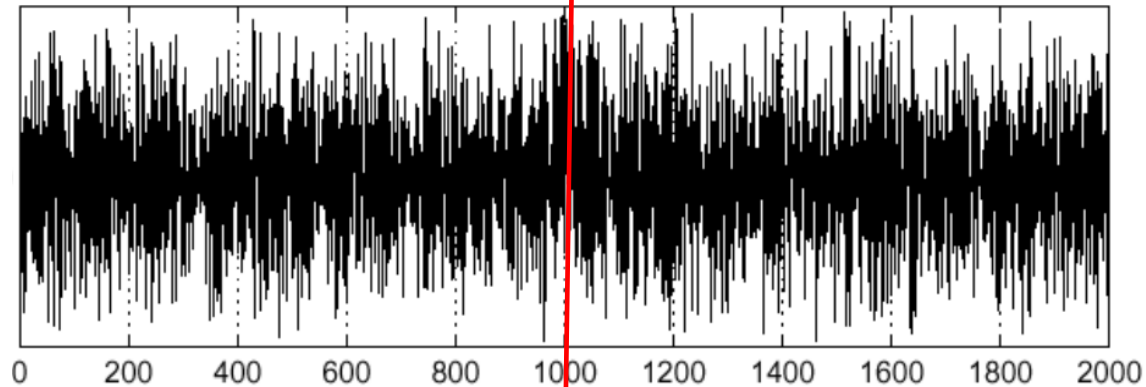
$\frac{\partial f}{\partial x}$

Gradient and Noise

$f(x)$

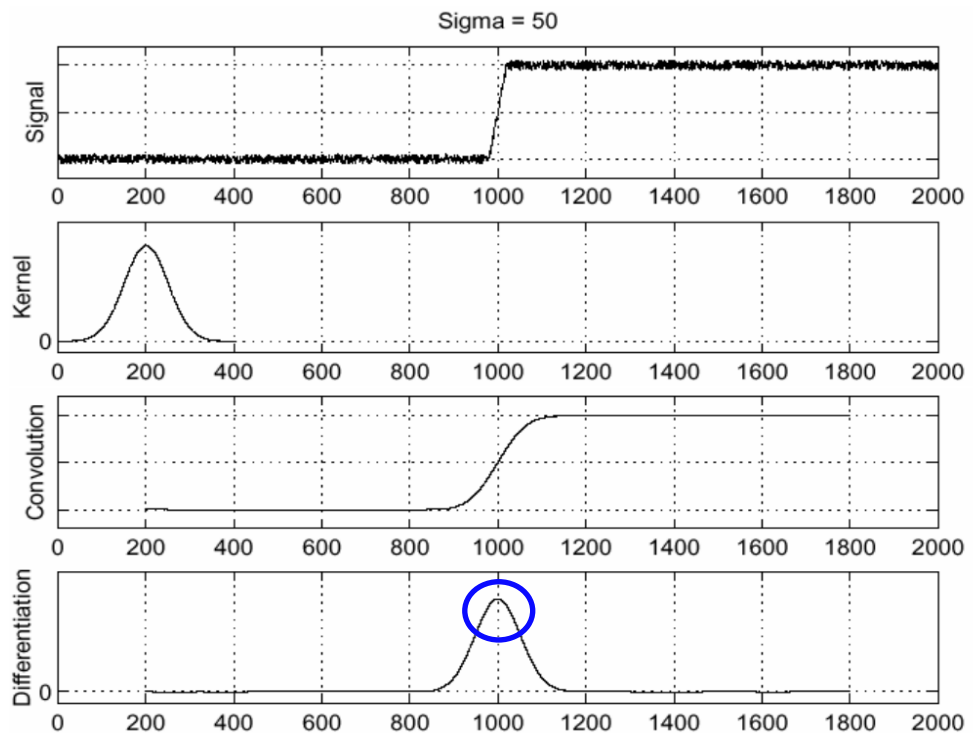


$\frac{d}{dx}f(x)$



- Derivative filter have a great sensitivity to noise
 - Noise $\rightarrow$ uniform areas are not so uniform
 - Pixels can be very different wrt neighbors
- Smoothing filtering allows to reduce noise effects

Smoothing & Derivative filtering



f

g

$f * g$

$\frac{d}{dx}(f * g)$

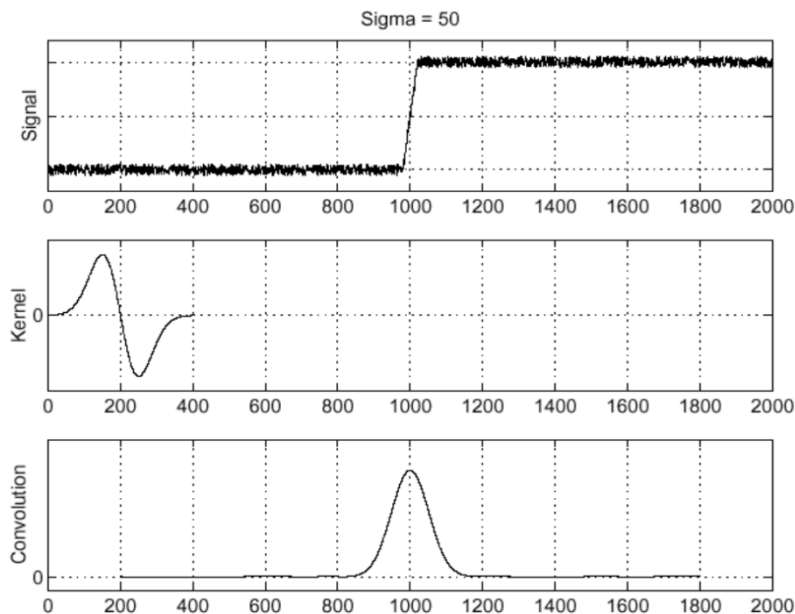
Edges correspond to peaks in
derivative of f and g
convolution

From: S. Seitz

Convolution properties

- Convolution is an associative operation
- We can optimize

$$\frac{\partial}{\partial x}(f * g) = f * \frac{\partial}{\partial x} g$$



f

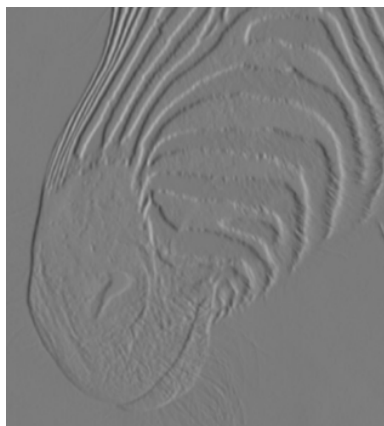
$\frac{d}{dx} g$

$f * \frac{d}{dx} g$

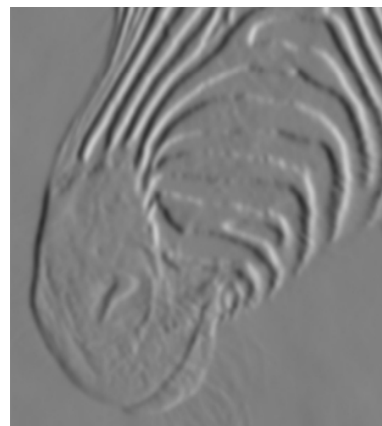
- Between smoothing and localization



Input image



$\sigma = 1px$



$\sigma = 3px$



$\sigma = 7px$

From: Forsyth & Ponce – *Computer Vision A Modern Approach*

Sobel Edge Detector

- Composition of a derivative operator and an averaging one
- Anyway a little rough for high frequency variations

-1	-2	-1
0	0	0
1	2	1

$$\begin{bmatrix} -1 \\ 0 \\ 1 \end{bmatrix} * \begin{bmatrix} 1 & 2 & 1 \end{bmatrix}$$

-1	0	1
-2	0	2
-1	0	1

$$\begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} * \begin{bmatrix} -1 & 0 & 1 \end{bmatrix}$$

Sobel Edge Detector: threshold

- Sobel operator allows to compute the gradient
 - Not actual edges
- We need a threshold



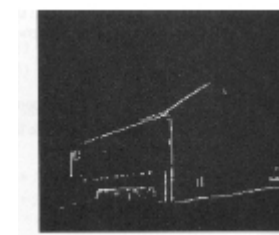
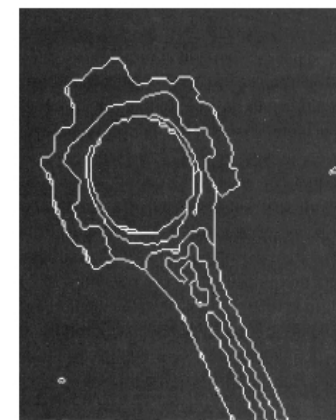
Sobel Edge Detector: edge thickness

- Each edge is “fat”



Canny Edge Detector

- Sobel is a quick&dirt solution with some issue:
 - Threshold
 - Fat edges
- There is a better approach?
- Yes → Canny Edge Detector
 - Gradient
 - Thinning
 - Thresholding



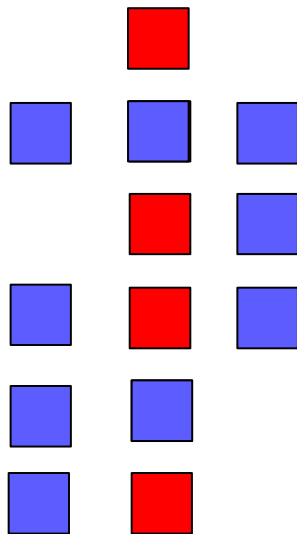
- Widely used
 - It performs very well!
- Criteria
 - Accuracy: reduce false positives (also negatives)
 - Localization: detected edges should match real edges as much as possible
 - Uniqueness: only one detection for each real edge

J. Canny, A Computational Approach to Edge Detection, IEEE Transactions on Pattern Analysis and Machine Intelligence, Vol 8, No. 6, Nov 1986

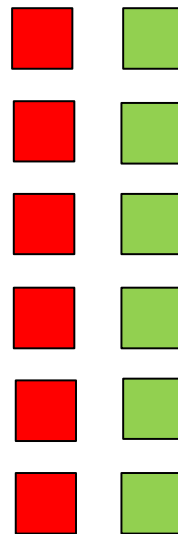
- Examples:



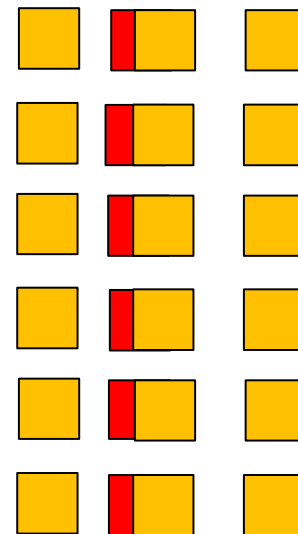
Real edge



Accuracy



Localization



Uniqueness

- 3 Main steps
 - Gradient detection: compute horizontal & vertical derivatives
 - Magnitude & Direction
 - Thinning: non local maxima suppression
 - 1 pixel wide gradients
 - Thresholding: hysteresis approach
 - Final result



- Sobel is ok!

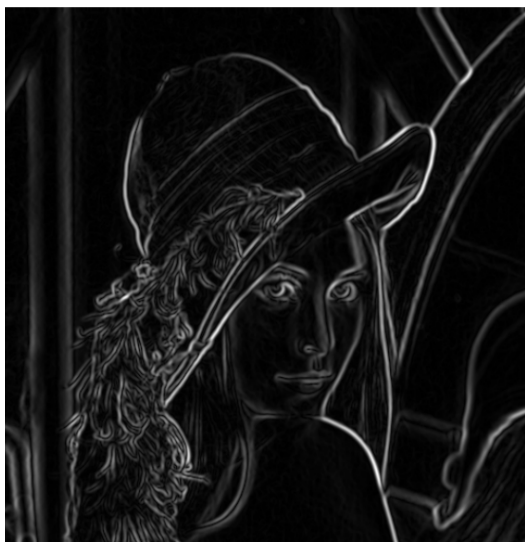


Horizontal Gradient G_x

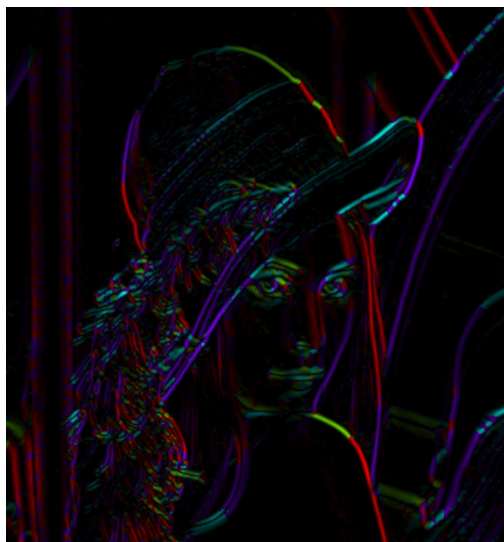


Vertical Gradient G_y

- We have 2 results: gradient magnitude and gradient direction



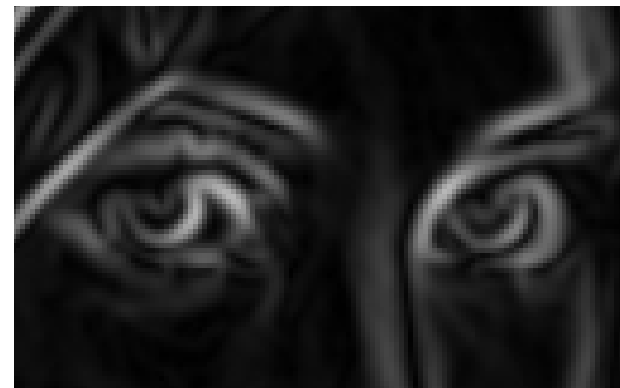
Magnitude



Orientation

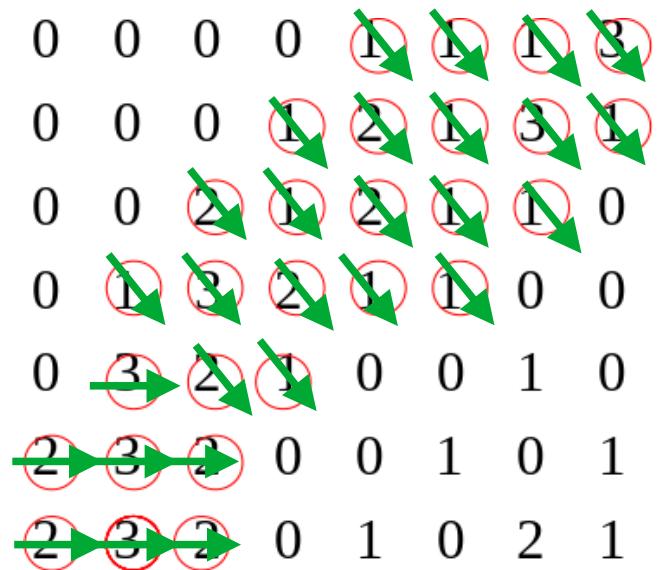
$$M = \sqrt{G_x^2 + G_y^2}$$

$$\Theta = \tan^{-1} \left(\frac{G_y}{G_x} \right) = \text{atan2}(G_y, G_x)$$



0	0	0	0	1	1	1	3
0	0	0	1	2	1	3	1
0	0	2	1	2	1	1	0
0	1	3	2	1	1	0	0
0	3	2	1	0	0	1	0
2	3	2	0	0	1	0	1
2	3	2	0	1	0	2	1

- Non Maxima Suppression
 - Only local maxima must survive
 - Orientation and gradient are used to detect local maxima
 - 2 approaches:
 - Bilinear interpolation or
 - A 3×3 grid

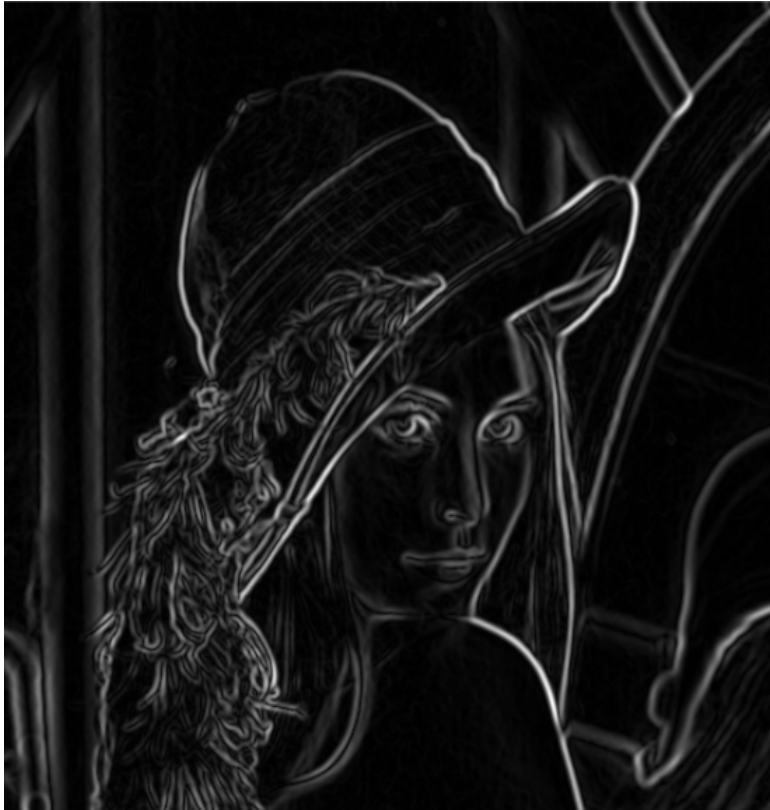


- Non Maxima Suppression
 - Only local maxima must survive
 - Orientation and gradient are used to detect local maxima
 - 2 approaches:
 - Bilinear interpolation or
 - A 3×3 grid



- Non Maxima Suppression
 - Only local maxima must survive
 - Orientation and gradient are used to detect local maxima
 - 2 approaches
 - Bilinear interpolation or
 - A 3×3 grid

Non Maxima Suppression





- Local Maxima does not mean “strong edges”
- One threshold is not enough
- Two thresholds:
 - Under T_{low} $\rightarrow$ eliminate edge
 - Over T_{high} $\rightarrow$ keep edge
 - Between T_{low} and T_{high} ?

- Edges above T_{high} are iteratively grown using adjacent edges above T_{low}
 - Similar to a labeling technique
- At the end edges $< T_{\text{high}}$ are removed

Hysteresis





UNIVERSITÀ DI PARMA

Lines Detection

- Edges and lines
- Hough transform
- Not only lines
 - Hough transform can be generalized

- Edges are not lines... still a low level feature

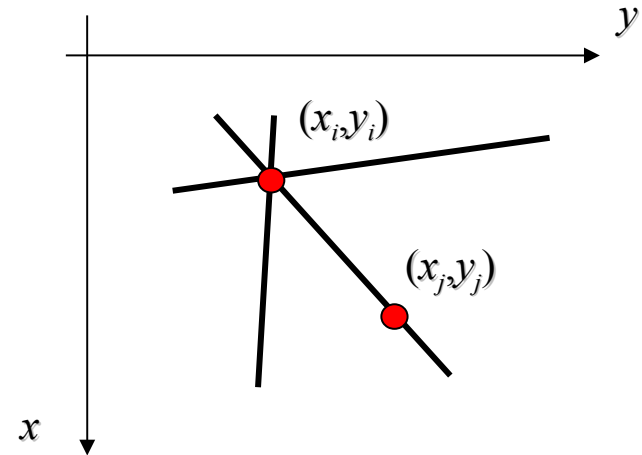


- First approach: determine pixel alignment
 - Expensive!
 - We have to match each pixel to other pixels $\rightarrow$ straight lines
 - Check whether other pixels are on those lines
 - For n pixels we have $n(n-1)/2 \sim n^2$ potential straight lines
 - $(n-2)[n(n-1)/2] \sim n^3$ matches
- Real approach: Hough Transform

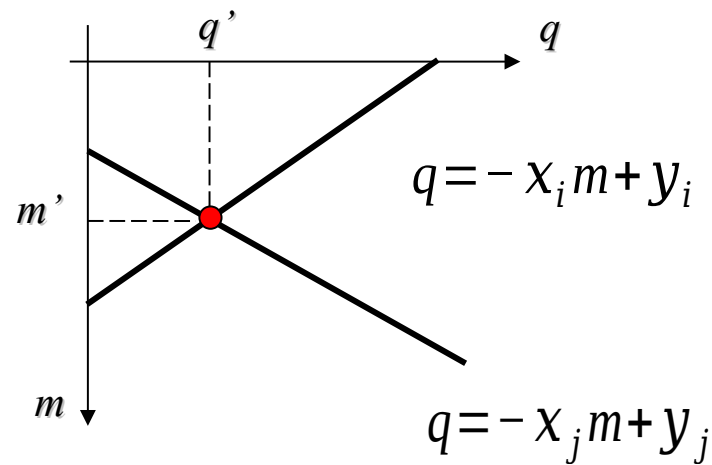
- The family of straight lines that pass through a given point (x_i, y_i) is defined as:

$$y_i = mx_i + q$$

- For each parameter set (m, q) we have a given straight line
- Actually x_i and y_i are constant, while m and q are the unknowns

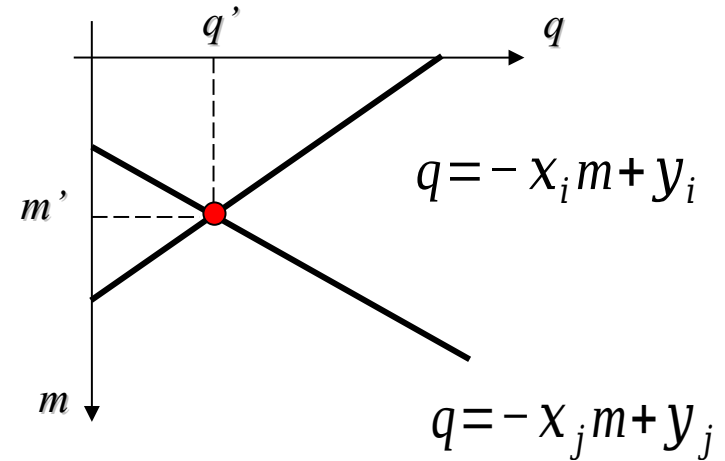


- Therefore, the previous equation can be written as:
$$q = -x_i m + y_i$$
- That represents all parameters values for m given point (x_i, y_i)
 - Still a straight line but in parameter space!
- This can be done also for other points, i.e. given point (x_j, y_j)
- The two straight lines intersect in (m', q')

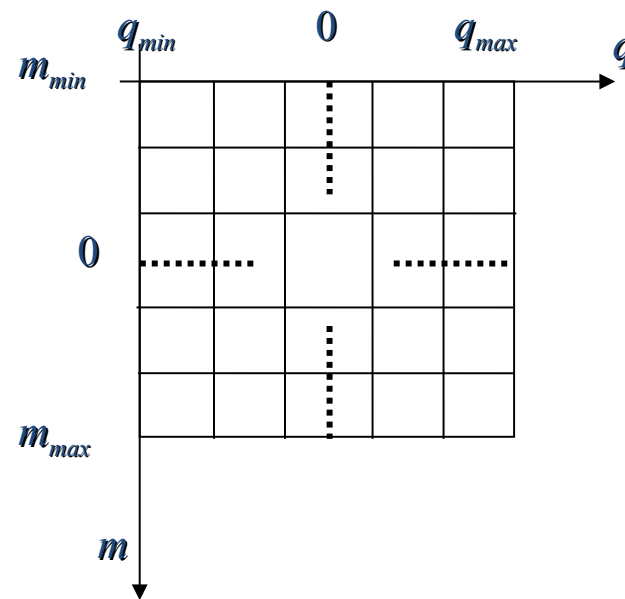


Hough Transform

- (m', q') are then the parameters that define the straight line that pass through both (x_i, y_i) and (x_j, y_j)
- More generally, all points that lie on the same line in the x, y space generate straight lines that intersect in a specific point in the m, q space
- This is the underlying rationale of the Hough Transform

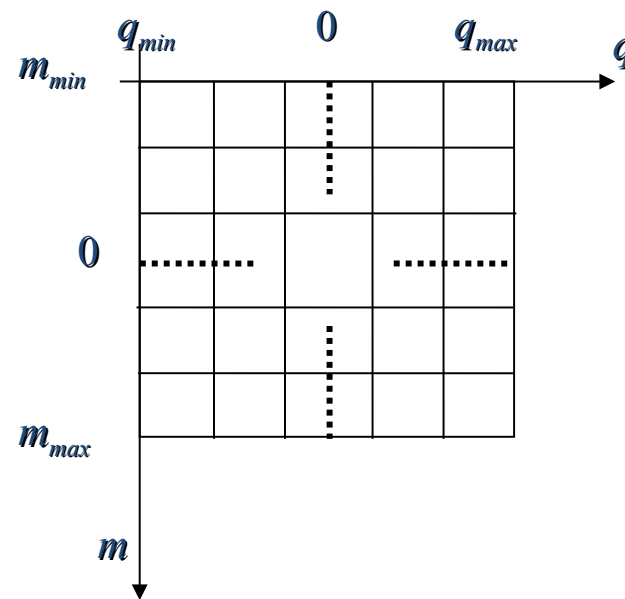


- Main steps:
 - Define a discrete accumulator matrix for the m, q space
 - For each (x_k, y_k) compute all possible q values using
 - All m values in the $m_{min} - m_{max}$ range
 - $q = -x_k m + y_k$
 - Increase the corresponding m, q cells in the accumulator matrix



Hough Transform

- At the end of the process each accumulation cell gives us information on how much pixels lie on the same straight line
- i.e. if a cell (m_i, q_i) contains the value M this means that in the image we have M points that lie on the $y=m_ix+q_i$ line
- A threshold is generally used to extract biggest values
 - Namely detected straight lines

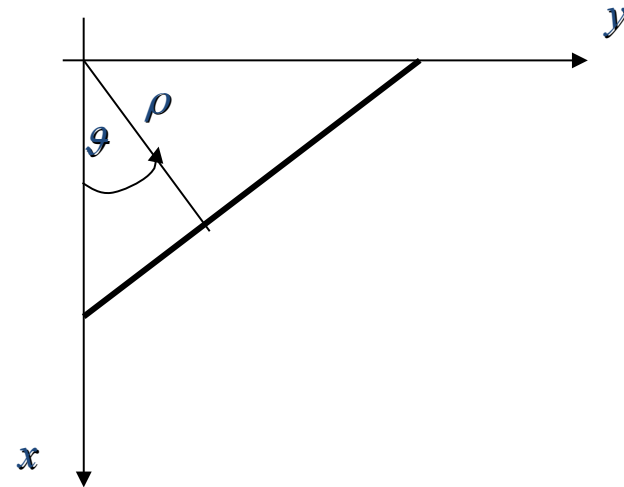


- If the m axis is subdivided in k intervals for each (x, y) point we get k values for q
- For n points we need to compute $n \times k$ values
 - $O(n)$ better than $O(n^3)$
- Open issues:
 - Which value for k ?
 - What range for m ?
 - Vertical lines?

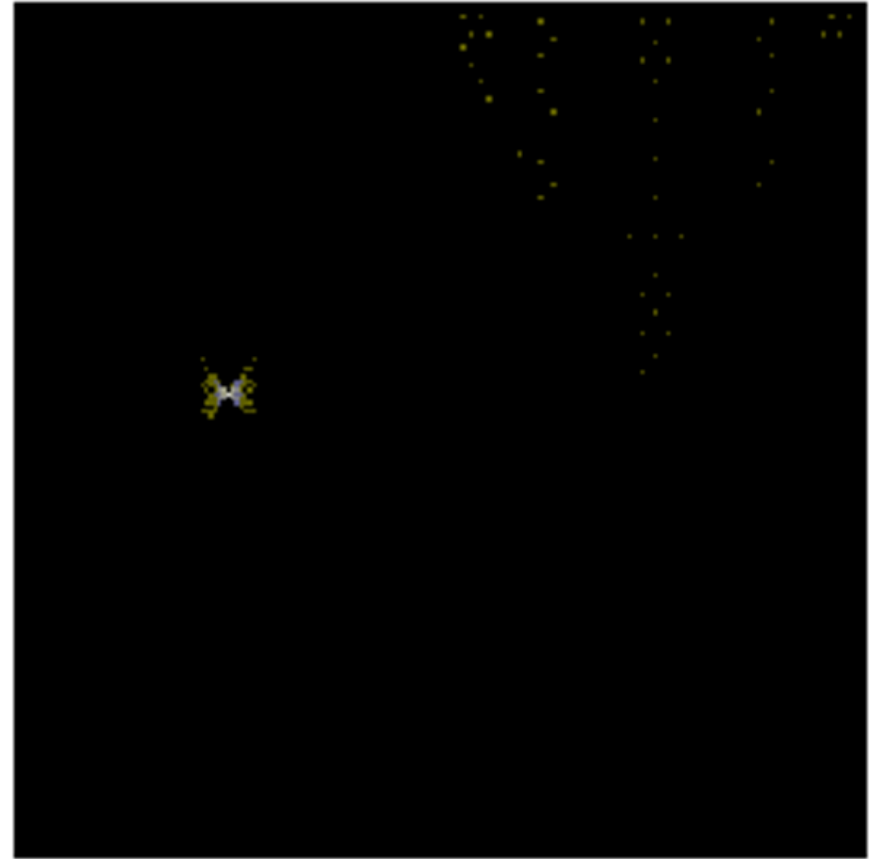
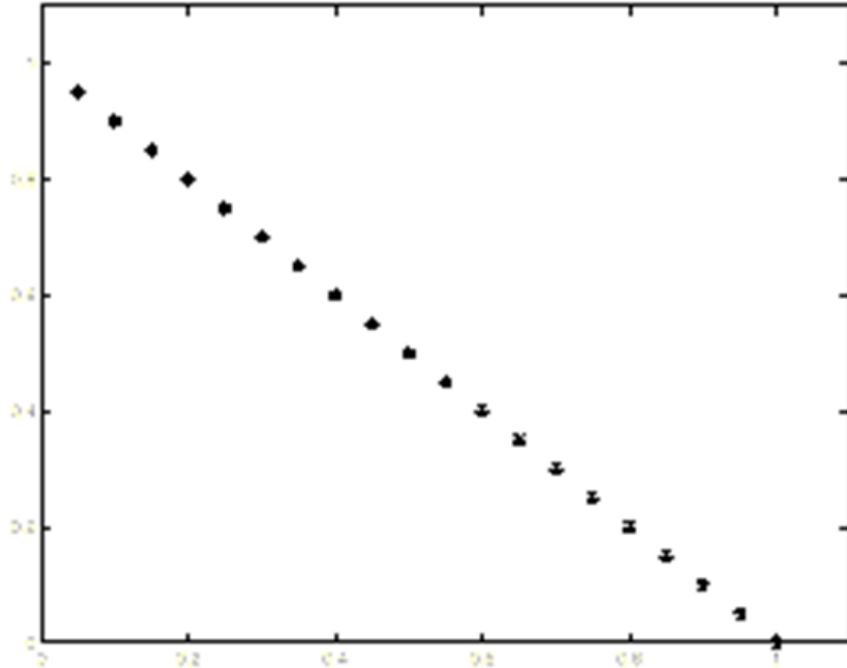
- To solve some issues Polar coordinates are used instead of Cartesian space

$$x \cos \theta + y \sin \theta = \rho$$

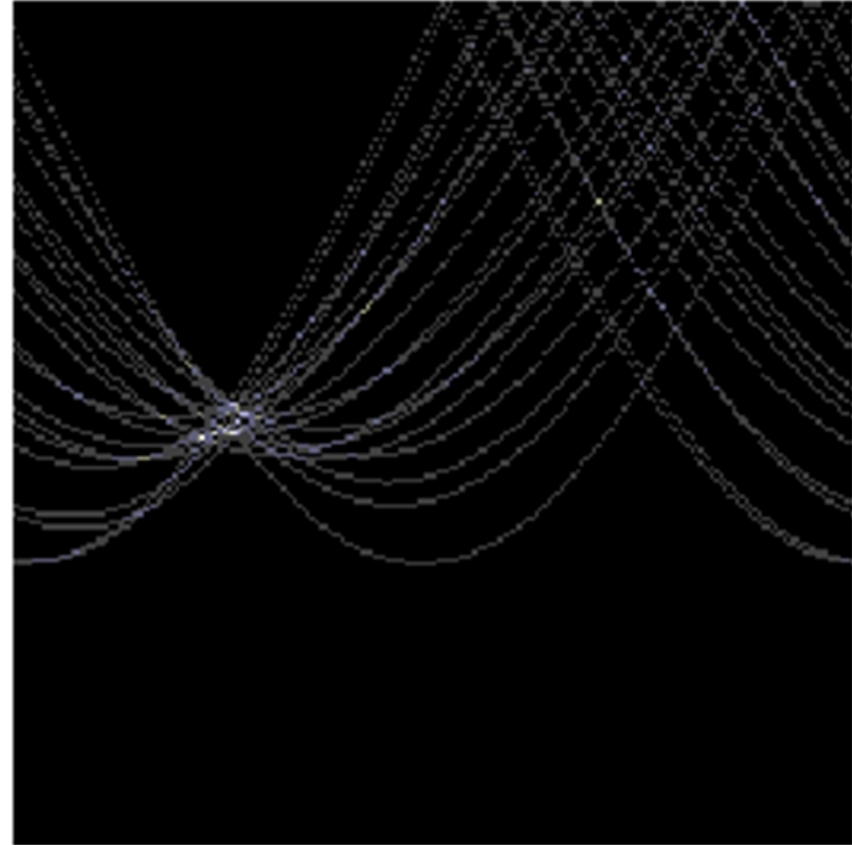
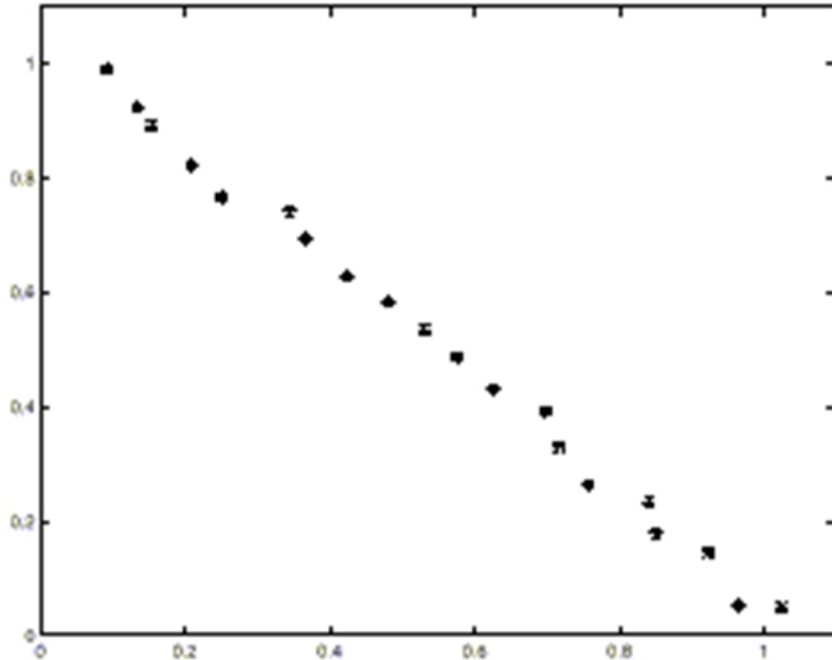
- Both parameters have a limited range
 - θ is an angle
 - ρ is limited by image diagonal
- Similar procedure as before
- In the parameters space we have curves



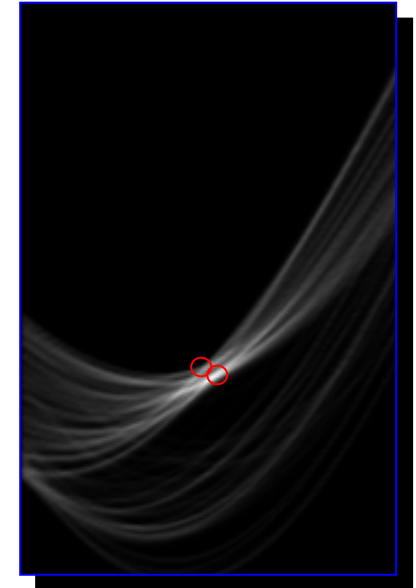
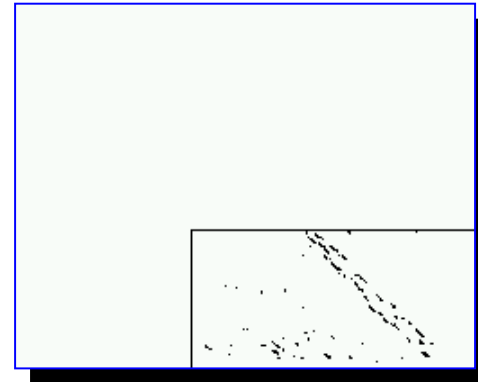
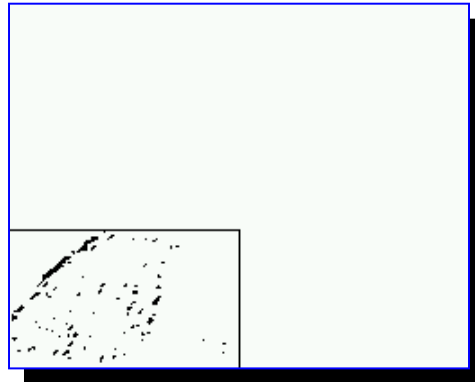
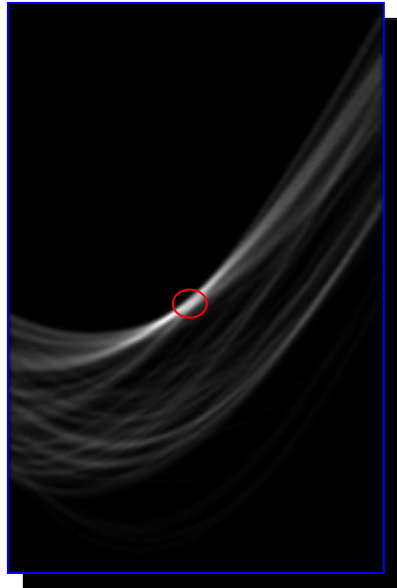
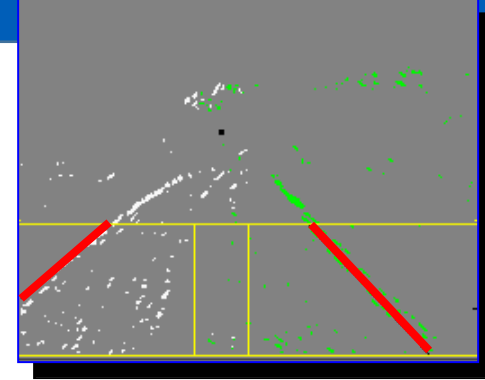
Hough Transform



Hough Transform



Hough Transform



- Described approach is for straight lines
- Anyway it can be applied to any analytical curve
- The only differences are the number and ranges of parameters
 - i.e. for a circle $(x-c_x)^2+(y-c_y)^2=r^2$
 - We need to use a 3D accumulator
 - Similar approach ($O(n^2)$)



UNIVERSITÀ DI PARMA

Edges Detection Lines Detection

Edges Detection
Lines Detection

- Edge detection
 - What is an edge?
 - Are they important?
 - Canny Edge Detector
- Lines
 - Hough Transform
 - Generalized Hough Transform

Sono due argomenti separati ma comunque legati tra di loro

In effetti quando si parla di edge detection siamo ancora a basso livello, il risultato è comunque una immagine

Quando estraiamo delle linee “usciamo” dallo spazio immagine. Il risultato è un elenco di “oggetti”. Poi per comodità li riproietteremo sull’immagine stessa.



UNIVERSITÀ DI PARMA

Edges Detection Lines Detection

Edges Detection

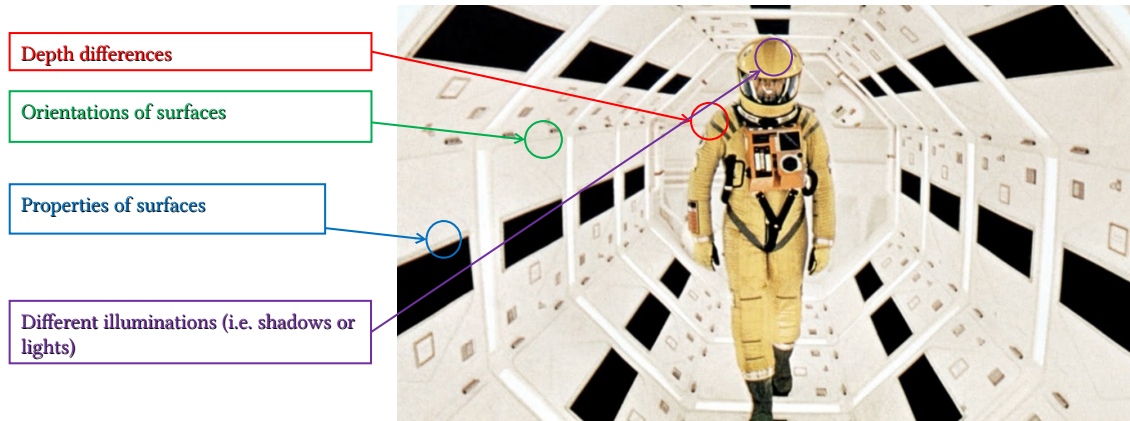


- Q: What is an edge? A: A discontinuity in intensity!



Prima di tutto chiediamo cosa sia un “bordo”. Di fatto è una discontinuità nei valori dei pixel. Quindi o di luminosità o di colore

- Q: From where intensity discontinuities (aka edges) come from?



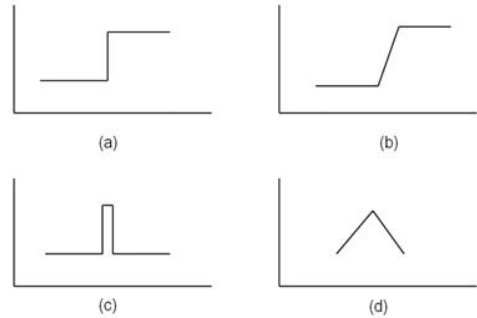
Diversi sono i motivi per cui io mi trovo tale discontinuità.

Per altro queste condizioni sono necessarie ma non sufficienti.

Ad esempio un oggetto che ha lo stesso colore dello sfondo, anche in presenza di salto di distanza, potrebbe non generare bordi....



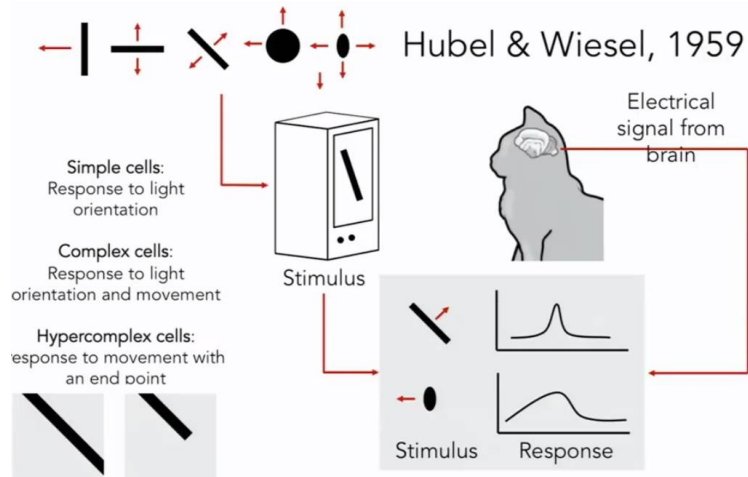
- (a) Step edge: abrupt change of intensity
- (b) Ramp edge: smooth change
- (c) Ridge/line edge: abrupt change followed by a return to starting value
- (d) Roof edge: a ridge edge with a smoother change



Inoltre gli edge possono presentare diverse forme, cosa che potrebbe anche cambiare le strategie di estrazione.

Ad esempio per le linee stradali si ricade nel (c). Per individuarle è possibile operare un filtraggio ad hoc.

- Why edges are important?

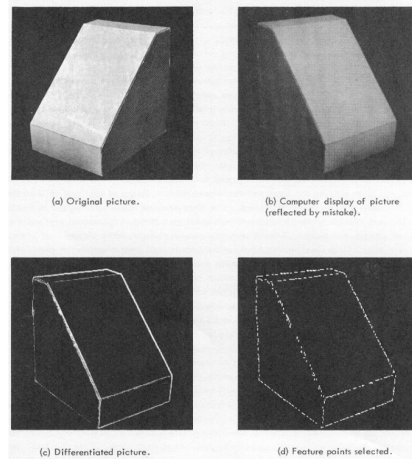


Ok ma perché sono importanti?

Esperimenti su animali hanno dimostrato come la sola presenza degli edge “triggera” determinate aree del cervello.



- Why edges are important?



Larry Roberts, 1963

In effetti l'oggetto in questa immagine è riconoscibilissimo anche solo dagli edge...



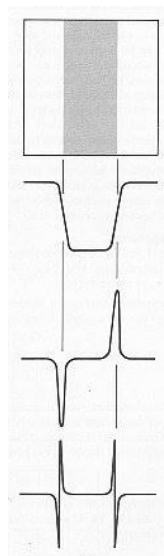
- Edges can help:
 - Extract information
 - Detect objects
 - Detect object shapes



- Already (partially) discussed
 - Actually we discussed about gradient extraction
 - And gradient $\neq$ edges
- Small recap:
 - Smoothing $\rightarrow$ to reduce noise
 - Derivative Horizontal & Vertical filtering $\rightarrow$ extract gradient

Abbiamo già parlato di radiente o meglio abbiamo visto alcuni filtri convolutivi in merito. Facciamo un piccolo recap

Attenzione che il gradiente mi fornisce informazioni su come cambia la luminosità ma gli edge sono solo i cambiamenti bruschi. Quindi ci vuole un altro passaggio...



Image

Horizontal Intensity

First derivative

Second derivative

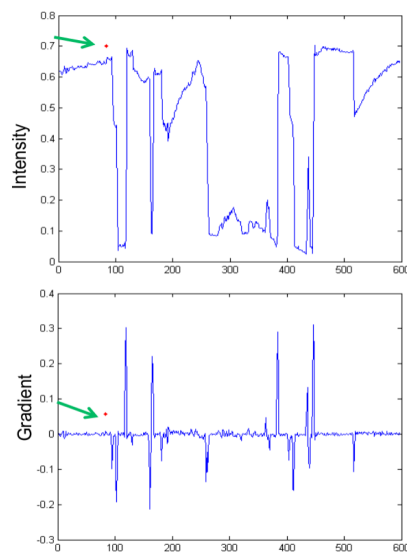
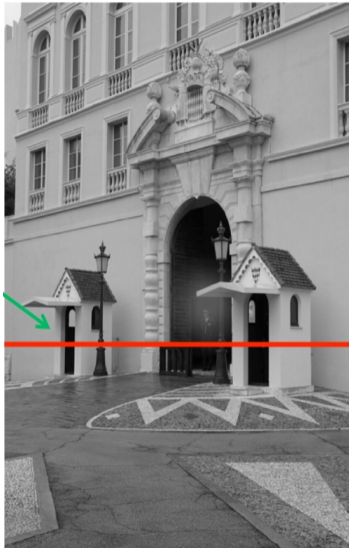
Edges are the maximum/minimum
in first derivative

Edges corresponds to 0 crossings in
second derivative

Se abbiamo un'immagine come quella qui sopra, sezionandola in orizzontale come cambia la sua luminosità? E cosa succede se derivo tale segnale?

Di fatto gli edge corrispondono a picchi nella derivata prima e a transizioni dallo 0 nella derivata seconda

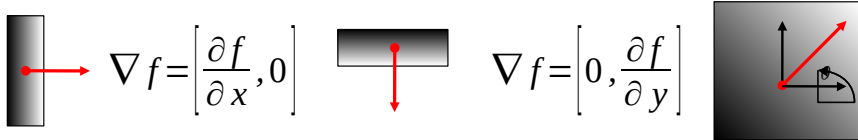
Gradient in real images



We have large peaks but also some noise

From: Fei Fei Li

- Gradient of image:



$$\nabla f = \left[\frac{\partial f}{\partial x}, 0 \right] \quad \nabla f = \left[0, \frac{\partial f}{\partial y} \right] \quad \nabla f = \left[\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y} \right]$$

- Gradient is a vector:

- Magnitude/Intensity
- Direction

$$\|\nabla f\| = \sqrt{\left(\frac{\partial f}{\partial x}\right)^2 + \left(\frac{\partial f}{\partial y}\right)^2}$$

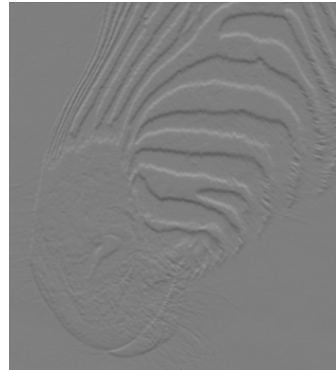
$$\theta = \tan^{-1}\left(\frac{\partial f}{\partial y} / \frac{\partial f}{\partial x}\right)$$

L'immagine è una funzione 2D. La derivata la ottengo combinando le due derivate (orizzontale e verticale).

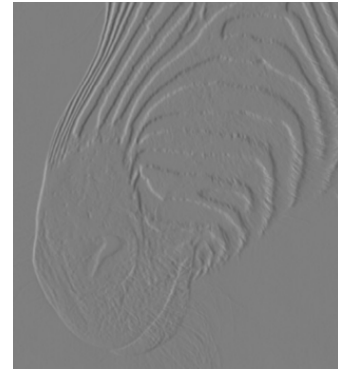
È un vettore quindi. Posso agevolmente calcolare la sua ampiezza (magnitude) e la direzione (occhio che la componente verticale è al numeratore)



f



$\frac{\partial f}{\partial y}$



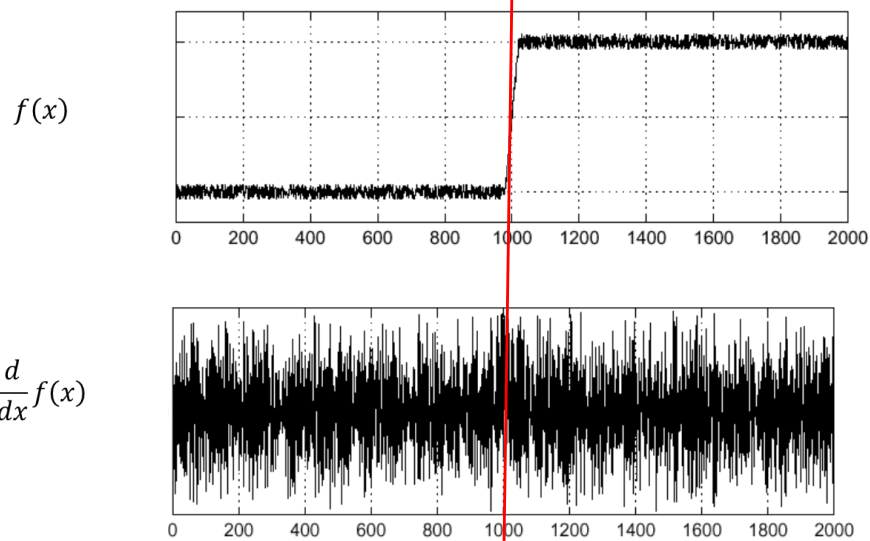
$\frac{\partial f}{\partial x}$

Forsyth & Ponce – *Computer Vision A Modern Approach*

Esempio di derivate orizzontali e verticali.

Per poterle rappresentare, dato che ho valori negativi, le riporto in positivo.

Quindi dove non ho bordi è quel grigio mentre le zone scure e chiare corrispondono alla presenza di bordo significativo.

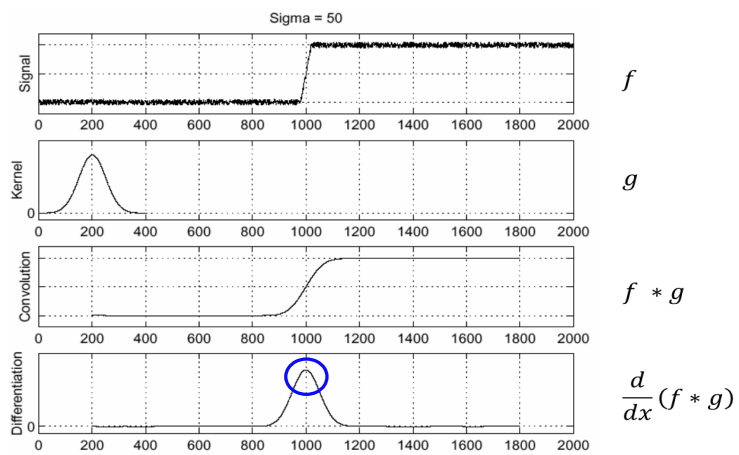


From: S. Seitz

Avevamo già visto come in generale fosse meglio fare un po' di smoothing.
Infatti derivare significa anche amplificare il rumore

- Derivative filter have a great sensitivity to noise
 - Noise $\rightarrow$ uniform areas are not so uniform
 - Pixels can be very different wrt neighbors
- Smoothing filtering allows to reduce noise effects

Smoothing & Derivative filtering



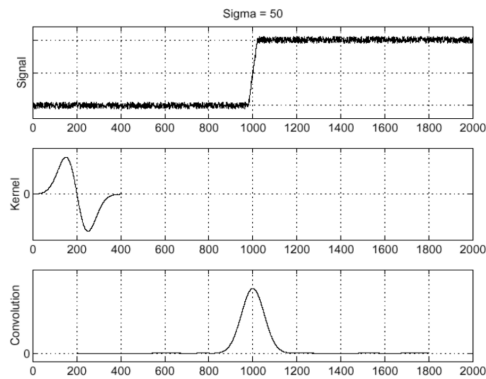
Edges correspond to peaks in
derivative of f and g
convolution

From: S. Seitz

Questo il risultato smoothando con un filtro gaussiano e poi derivando.

- Convolution is an associative operation
- We can optimize

$$\frac{\partial}{\partial x}(f * g) = f * \frac{\partial}{\partial x} g$$



f

$\frac{d}{dx}g$

$f * \frac{d}{dx}g$

From: S. Seitz

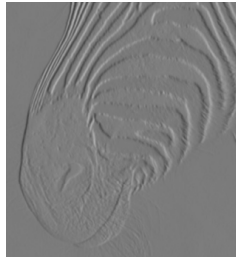
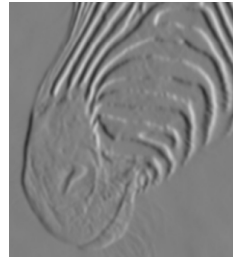
Si può ottimizzare usando la derivata del gaussiano

A parte saltare un passaggio, derivare un kernel che è tipicamente piccolo è più facile

- Between smoothing and localization



Input image

 $\sigma = 1px$  $\sigma = 3px$  $\sigma = 7px$

From: Forsyth & Ponce – *Computer Vision A Modern Approach*

The scale (i.e., σ) of the Gaussian used in a derivative of Gaussian filter has significant effects on the results. The three images show estimates of the derivative in the x direction of an image of the head of a zebra obtained using a derivative of Gaussian filter with σ one pixel, three pixels, and seven pixels (left to right). Note how images at a finer scale show some hair, the animal's whiskers disappear at a medium scale, and the fine stripes at the top of the muzzle disappear at the coarser scale

- Composition of a derivative operator and an averaging one
- Anyway a little rough for high frequency variations

-1	-2	-1
0	0	0
1	2	1

$$\begin{bmatrix} -1 \\ 0 \\ 1 \end{bmatrix} * \begin{bmatrix} 1 & 2 & 1 \end{bmatrix}$$

-1	0	1
-2	0	2
-1	0	1

$$\begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} * \begin{bmatrix} -1 & 0 & 1 \end{bmatrix}$$

Senza scomodare i gaussiani, il filtro di Sobel permette di ottenere risultati comunque apprezzabili. Di fatto combina una media letterbox con una derivata “secca”.

- Sobel operator allows to compute the gradient
 - Not actual edges
- We need a threshold



Che io usi Sobel o il precedente operatore non ottengo affatto gli edge.

Ottingo un gradiente.

Ma sappiamo come binarizzare una immagine? Sì! → applico una soglia

- Each edge is “fat”



Questo è il risultato.

Accettabile ma quasi sempre l'edge è spesso qualche pixel



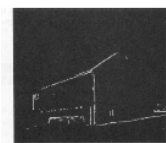
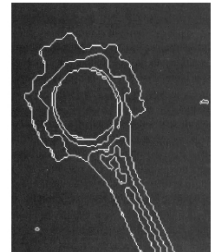
- Sobel is a quick&dirt solution with some issue:

- Threshold
- Fat edges

- There is a better approach?

- Yes → Canny Edge Detector

- Gradient
- Thinning
- Thresholding



E poi come la trovo la soglia?

Otsu non funziona benissimo

C'è di meglio? Oh yes! Il mitico filtro di Canny!



- Widely used
 - It performs very well!
- Criteria
 - Accuracy: reduce false positives (also negatives)
 - Localization: detected edges should match real edges as much as possible
 - Uniqueness: only one detection for each real edge

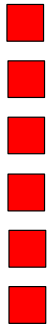
J. Canny, A Computational Approach to Edge Detection, IEEE Transactions on Pattern Analysis and Machine Intelligence, Vol 8, No. 6, Nov 1986

I presupposti sono:

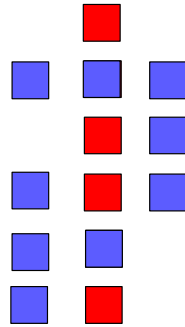
- essere preciso, voglio ricattare gli edge corretti (da questo punto di vista Sobel non è comunque malaccio)
- L'edge che ricavo deve trovarsi esattamente dove è nell'immagine
- no edge cicciotti, voglio una linea sul punto esatto



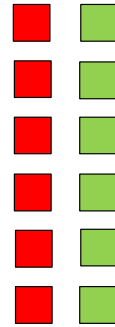
- Examples:



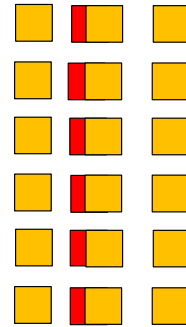
Real edge



Accuracy



Localization



Uniqueness

- 3 Main steps
 - Gradient detection: compute horizontal & vertical derivatives
 - Magnitude & Direction
 - Thinning: non local maxima suppression
 - 1 pixel wide gradients
 - Thresholding: hysteresis approach
 - Final result

Chiaramente leggermente più complicato di un Sobel + soglia

3 passi fondamentali, vediamo uno alla volta



Usiamo la solita Lenna (nota di colore, da qualche anno molte conferenze e riviste scientifiche hanno iniziato a proibire l'uso di questa immagine in quanto considerata degradante per il mondo femminile)



- Sobel is ok!



Horizontal Gradient G_x

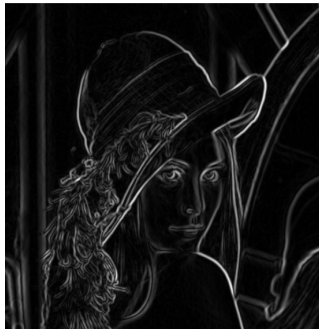


Vertical Gradient G_y

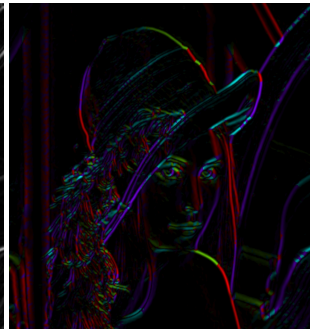
Per il calcolo del gradiente Sobel va benissimo (quick & dirt)



- We have 2 results: gradient magnitude and gradient direction



Magnitude



Orientation

$$M = \sqrt{G_x^2 + G_y^2}$$

$$\Theta = \tan^{-1} \left(\frac{G_y}{G_x} \right) = \text{atan2}(G_y, G_x)$$



Non sogliamo il risultato e quindi quanto otteniamo è il gradiente sbiotto e blurrato

0	0	0	0	1	1	1	3
0	0	0	1	2	1	3	1
0	0	2	1	2	1	1	0
0	1	3	2	1	1	0	0
0	3	2	1	0	0	1	0
2	3	2	0	0	1	0	1
2	3	2	0	1	0	2	1

- Non Maxima Suppression

- Only local maxima must survive
- Orientation and gradient are used to detect local maxima
- 2 approaches:
 - Bilinear interpolation or
 - A 3×3 grid

Oltre a non essere sogliaato è sempre “ciccio”

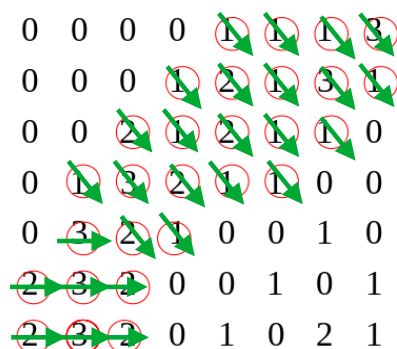
Uso una NMS per assottigliarlo. Vince l’edge più alto.

Quindi uso sicuramente la magnitude del gradiente

Nell’immagine qui sopra un pezzettino (valori un po’ a caso, infatti sono molto bassi, ma è per far capire il meccanismo)

- Non Maxima Suppression

- Only local maxima must survive
- Orientation and gradient are used to detect local maxima
- 2 approaches:
 - Bilinear interpolation or
 - A 3×3 grid



Il più alto vince, ok. Ma con chi li confronto? Con quelli vicini

Ma vicini non a caso ma lungo la direzione del gradiente stesso.

Nell'immagine qui sopra abbiamo direzioni già quantizzate (verticali, orizzontali o "diagonali"). In questo caso è facile so che mi devo confrontare con quelli di fianco o sopra/sotto o in diagonale in un vicinato 3x3.

Azzero quelli che hanno un vicino "più grande".

Già, ma col cavolo che io otterrò direzioni di questo tipo, saranno quasi sempre valori diversi. Banalmente arrotondo quindi se ottengo una direzione di $-72,18^\circ$ la faccio diventare -90°

In realtà spesso si applica una interpolazione bilineare (la vedremo in laboratorio!) che fornisce risultati migliori. Il succo è usare l'angolo per calcolare esattamente le coordinate dei due pixel vicini. Otterrò però coordinate non intere. Una interpolazione bilineare mi permette di calcolare il valore fittizio di un pixel a coordinate non intere (mi ripeto, ci torneremo...)

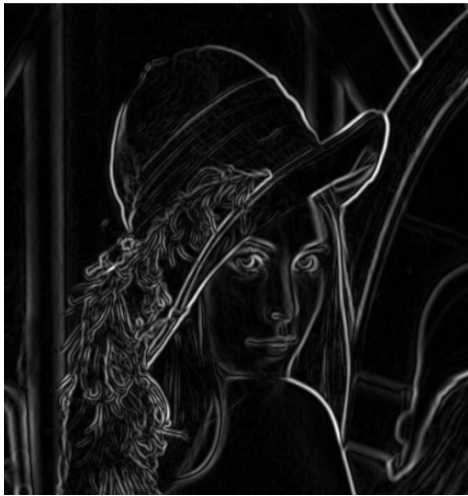


- Non Maxima Suppression

- Only local maxima must survive
- Orientation and gradient are used to detect local maxima
- 2 approaches
 - Bilinear interpolation or
 - A 3×3 grid



La NMS non sopprime il rumore come ad esempio edge isolati



Questo il risultato. Il gradiente non ha più aree cicciotte

Però non ho soglia ancora nulla quindi posso vedere linee belle marcate e altre appena percettibili



- Local Maxima does not mean “strong edges”
- One threshold is not enough
- Two thresholds:
 - Under T_{low} $\rightarrow$ eliminate edge
 - Over T_{high} $\rightarrow$ keep edge
 - Between T_{low} and T_{high} ?

Non applico però una soglia secca ma ne uso due

Tutto ciò che è al di sopra della soglia alta lo metto a 1 (cioè 255)

Tutto ciò che è sotto lo azzerò.

E quanto è in mezzo?



- Edges above T_{high} are iteratively grown using adjacent edges above T_{low}
 - Similar to a labeling technique
- At the end edges $< T_{\text{high}}$ are removed

In pratica faccio “crescere” gli edge messi a 255

Quindi gli edge intermedi se sono contigui ad aree già sopra la soglia alta vengono tenuti anche loro, se no li sopprimo



Risultato finale



UNIVERSITÀ DI PARMA

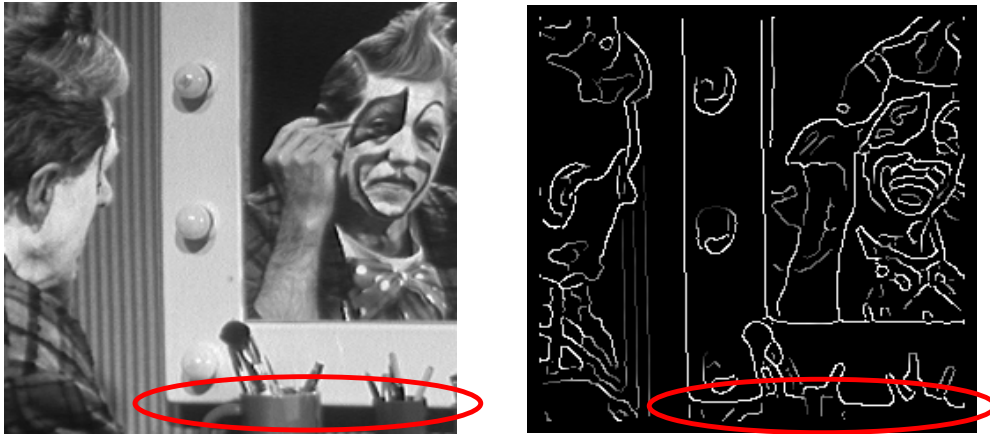
Lines Detection



Lines Detection

- Edges and lines
- Hough transform
- Not only lines
 - Hough transform can be generalized

- Edges are not lines... still a low level feature



Abbiamo appena visto il filtro di Canny, buon risultato ma tuttora una immagine. Ancora elaborazione immagini.

Vorrei estrarre delle caratteristiche dell'immagine stessa, ad esempio i segmenti di retta che ci sono. Il risultato a questo punto non è più una immagine ma un elenco di "linee"

Poi magari per rappresentarle uso ancora le immagini (ma solo a livello grafico)

- First approach: determine pixel alignment
 - Expensive!
 - We have to match each pixel to other pixels → straight lines
 - Check whether other pixels are on those lines
 - For n pixels we have $n(n-1)/2 \sim n^2$ potential straight lines
 - $(n-2)[n(n-1)/2] \sim n^3$ matches
- Real approach: Hough Transform

Capire se ho pixel allineati è semplice. Considero tutte le coppie di pixel possibili. Traccio la retta tra di loro e guardo se su quella retta ne cadono altri (tenendo magari conto della quantizzazione).

Funzionerebbe, ma è un po' costoso a livello computazionale.

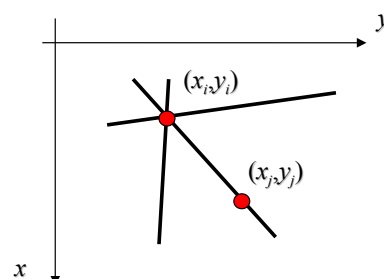
Esiste però qualcosa di meno oneroso: la trasformata di Hough



- The family of straight lines that pass through a given point (x_i, y_i) is defined as:

$$y_i = mx_i + q$$

- For each parameter set (m, q) we have a given straight line
- Actually x_i and y_i are constant, while m and q are the unknowns



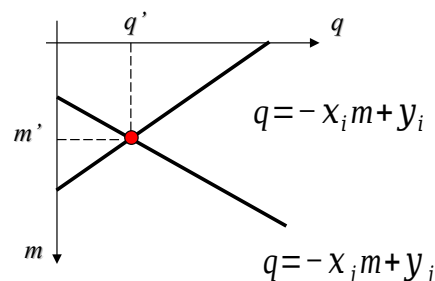
Dato un punto x_i, y_i nello spazio cartesiano il fascio di rette che passano da quel punto lo posso vedere come $y_i = mx_i + q$

Al variare di m e q individuo una delle rette del fascio

Anzi va detto che, visto che il punto è noto, x_i e y_i sono fissi. Quello che cambia sono solo m e q . Quindi se devo trovare una ben precisa retta di quel fascio sono loro due le incognite e non x_i e y_i

Riscriviamo quindi quella equazione

- Therefore, the previous equation can be written as:
 $q = -x_i m + y_i$
- That represents all parameters values for m given point (x_i, y_i)
 - Still a straight line but in parameter space!
- This can be done also for other points, i.e. given point (x_j, y_j)
- The two straight lines intersect in (m', q')



E cambiamo anche spazio cartesiano visto che a variare sono m e q (lo so che sembra controintuitivo, siamo troppo abituati a x e y come lettere per le incognite).

Quella equazione mi rappresenta comunque una retta anche nel nuovo spazio cartesiano. Tutti gli m e q possibili delle rette che passano per x_i e y_i noti

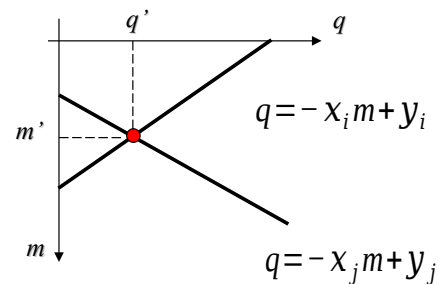
Se facciamo la stessa cosa con un secondo punto di coordinate note x_j e y_j otterremmo una seconda retta.

Possibile che le due rette si intersichino (non succede solo se $x_i = x_j$ perché le due rette avrebbero la stessa inclinazione in questo piano cartesiano ma soprattutto perché i due punti sarebbero verticalmente uno sull'altro e l'equazione di partenza non gestisce rette verticali)

Comunque se si intersecano lo faranno in un punto di quello spazio che indichiamo (m', q')

Cosa mi indica quel punto? I parametri m e q di una retta che passa sia per x_j, y_j che per x_i, y_i

- (m', q') are then the parameters that define the straight line that pass through both (x_i, y_i) and (x_j, y_j)
- More generally, all points that lie on the same line in the x, y space generate straight lines that intersect in a specific point in the m, q space
- This is the underlying rationale of the Hough Transform

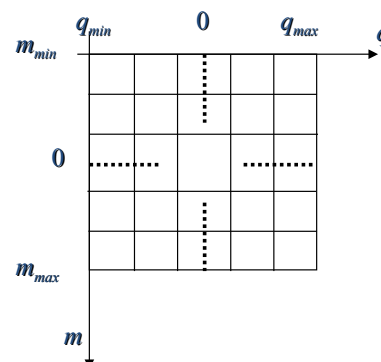


Interessante, quindi se tanti punti mi generano rette che passano per m', q' vuol dire che ho una linea.

Quindi posso, per ciascun punto calcolare quelle rette e i relativi punti di intersezione e dove mi si intersecano tante ho punti allineati

Sembra complesso ma in realtà è relativamente semplice da farsi

- Main steps:
 - Define a discrete accumulator matrix for the m, q space
 - For each (x_k, y_k) compute all possible q values using
 - All m values in the $m_{min} - m_{max}$ range
 - $q = -x_k m + y_k$
 - Increase the corresponding m, q cells in the accumulator matrix



Mi creo una matrice dei possibili m e q .

In pratica considero un minimo e un massimo per i possibili valori di q e m (questa scopriremo essere il punto debole dell'approccio) e tra i valori di minimo e massimo quantizzo (altrimenti dovrei considerare infiniti valori).

Ad esempio ipotizzando che q vari da -480 a +480 magari considero solo i q interi o anche solo quelli pari. Dipende un po' da quanto posso essere preciso.

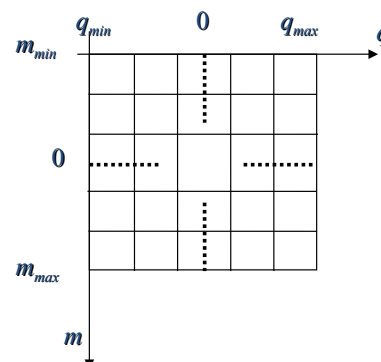
Riempio quella matrice di 0

Poi ci sono un paio di cicli nidificati da fare:

1. considero tutti i punti (primo ciclo)
2. per ciascuno di questi considero i possibili valori di m ovvero nuovo ciclo da m_{min} a m_{max} .
3. Per ogni tripletta di x, y, m mi calcolo il q in base a quella formula
4. m e q mi identificano una ben precisa cella di quella matrice, ne incremento il contenuto di 1



- At the end of the process each accumulation cell gives us information on how much pixels lie on the same straight line
- i.e. if a cell (m_i, q_i) contains the value M this means that in the image we have M points that lie on the $y = m_i x + q_i$ line
- A threshold is generally used to extract biggest values
 - Namely detected straight lines



Alla fine del procedimento in ogni cella di quella matrice di coordinate m e q avrò l'indicazione di quanti punti si trovino sulla retta definita da quei valori di m e q .

Quindi se in una cella trovo 123 vuol dire che su quella retta si trovano 123 punti.

Pochi? Molti? Dipende da cosa cerco, dalle dimensioni dell'immagine ecc.

Comunque sia userò una soglia (ed eventualmente un NMS) per isolare gli m e q che descrivono rette che passano da un numero di punti significativamente grande

- If the m axis is subdivided in k intervals for each (x, y) point we get k values for q
- For n points we need to compute $n \times k$ values
 - $O(n)$ better than $O(n^3)$
- Open issues:
 - Which value for k ?
 - What range for m ?
 - Vertical lines?

È efficiente questo approccio?

Beh se l'intervallo dei valori possibili di m è diviso in k elementi, dato n il numero di pixel da capire se sono allineati o meno mi occorrerà fare kn passi.

Complessità $O(n)$ insomma che è già meglio dell'approccio "provo tutti con tutti"

Bello ma non funziona benissimo

I problemi maggiori sono (la quantizzazione è secondaria) quale intervallo scelgo per m ? (e in realtà anche per q volendo). Per rette reali m può andare da $-a$ a $+\text{Inf}$, non comodissimo da gestire.

E poi il sistema non mi permette di gestire linee verticali (che di solito ci sono eh?)

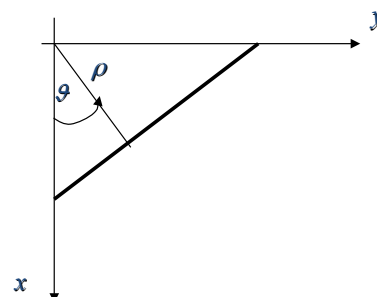
Infatti questo approccio non si usa mai. O meglio l'idea si usa ma applicata non al piano cartesiano bensì usando le coordinate polari!



- To solve some issues Polar coordinates are used instead of Cartesian space

$$x \cos \theta + y \sin \theta = \rho$$

- Both parameters have a limited range
 - θ is an angle
 - ρ is limited by image diagonal
- Similar procedure as before
- In the parameters space we have curves



Quella è l'equazione di una retta nel in coordinate polari

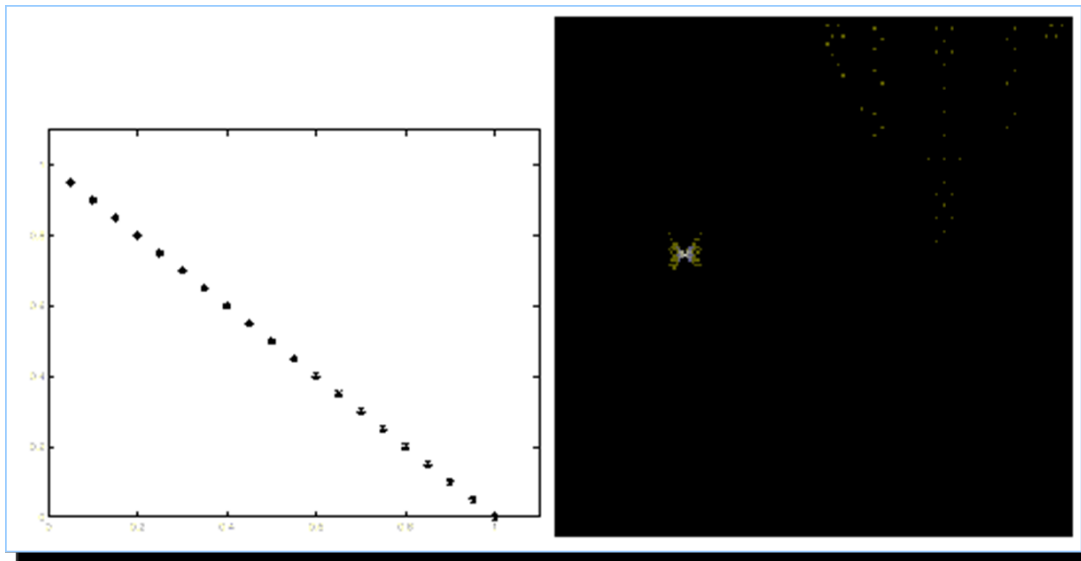
A questo punto le mie incognite diventano teta e ro.

Se considero lo spazio con assi teta e ro quella equazione non è più quella di una retta (machissene)

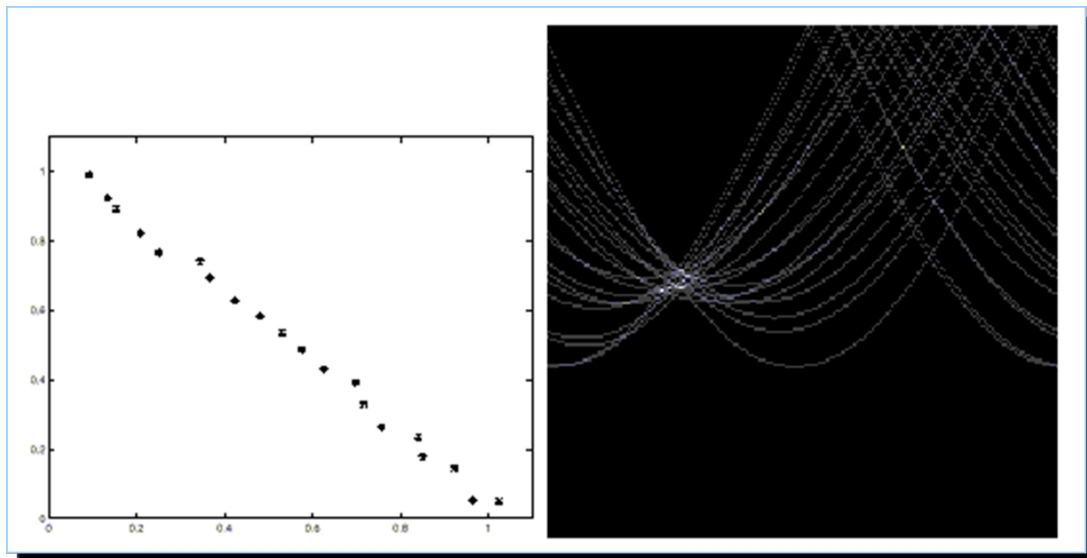
Anche qui posso comunque fare una matrice ecc.

Vantaggio enorme: entrambi i parametri sono limitati di solito teta lo limito tra 0 e 180° e ro tra + e meno la diagonale. Oppure considero tutti i 360 per l'angolo e da 0 alla diagonale per ro

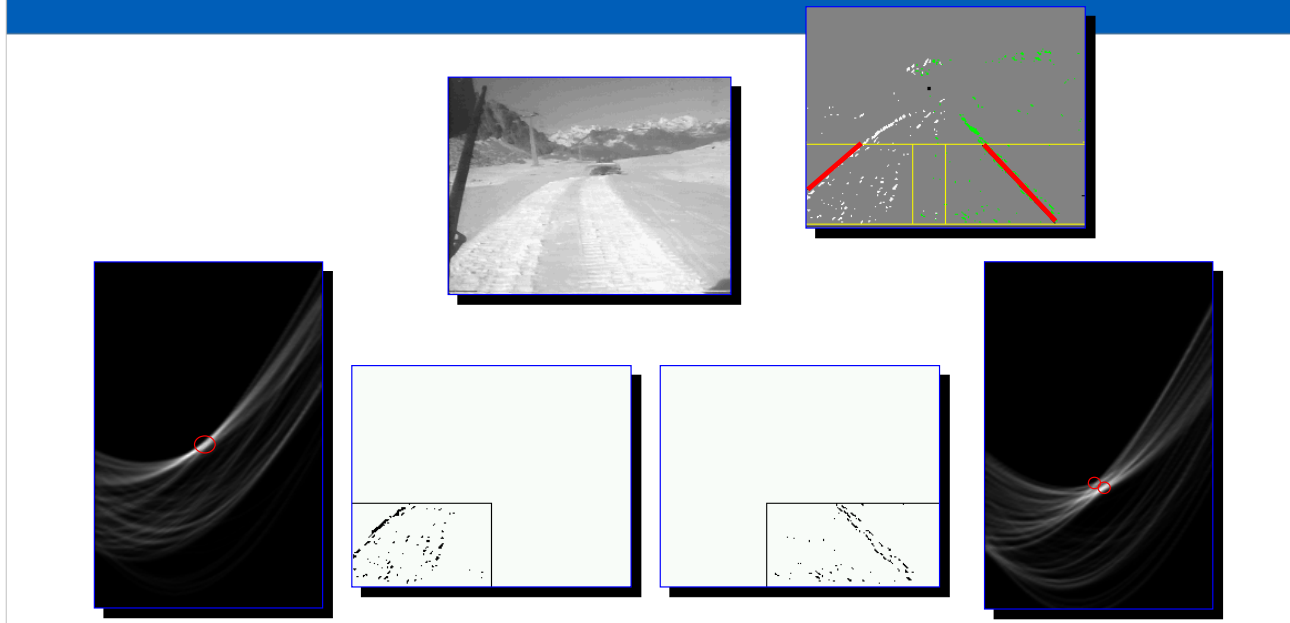
Ma comunque sia la mia matrice adesso ha dimensioni finite (devo comunque scegliere quanto quantizzare ma è il solito compromesso efficienza-precisione e poi per ro va detto che numeri interi vanno già bene)



Esempio con punti belli allineati. Ottengo una matrice praticamente di 0 se non intorno a un ben preciso valore di θ e ρ



Se aggiungo un po' di perturbazione si vede meglio che nello spazio teta/ro non ottengo rette. Ma comunque un massimo locale lo riesco a trovare



Esempio reale, trovare le tracce di un gatto delle nevi in Antartide (le immagini sono in realtà del Tonale).

Si estraggono in una sola zona le tracce e si ricavano gli edge (un semplice Sobel + soglia).

Ai lati sono mostrate le matrici di accumulazione di cui si individuano i massimi (mi serve una sola linea nelle due aree) e la ridisegno sull'immagine originale

- Described approach is for straight lines
- Anyway it can be applied to any analytical curve
- The only differences are the number and ranges of parameters
 - i.e. for a circle $(x-c_x)^2+(y-c_y)^2=r^2$
 - We need to use a 3D accumulator
 - Similar approach ($O(n^2)$)

La trasformata di Hough è abbastanza versatile, la posso estendere a tutte quelle forme/curve descrivibili tramite equazione (ad esempio cerchi).

Chiaro che all'aumentare dei parametri non avrò una matrice solamente bidimensionale e anche un impatto sul costo computazionale